

Auto battling Dapp

Whitepaper

Marshall Inch | Track B

1. The problem

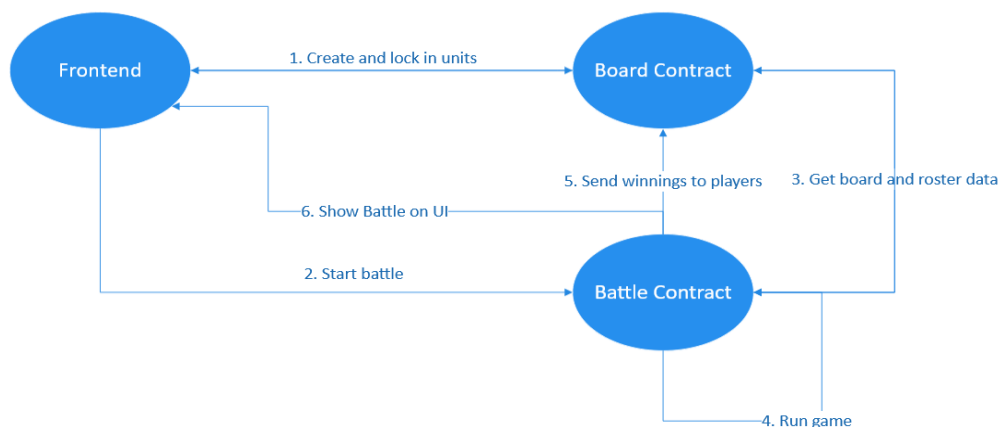
Online competitive games require trust within the central server and the company that owns it. A game company can silently alter unit balance, manipulate matchmaking, or withhold payouts with no recourse for players. Meaning that the guarantees players believe that they have are not certain.

For auto-battler type games the outcome is determined by the units selected and the draft they receive, this allows manipulation from the server to disadvantage players. This proof of concept replaces the trusted server with two NEAR Protocol smart contracts that guarantees:

- Game rules are encoded in immutable, publicly auditable contract code
- Random shop rolls are derived from a player-supplied seed verified on-chain
- Battle outcomes are computed deterministically and recorded permanently
- Rewards are transferred directly to winners' wallets with no intermediary

2. Architecture

The system consists of two smart contracts and a React frontend



The Board Registry contract owns all player state and the unit roster, organised into seasons. The Battle Contract owns battle state and calls the Board Registry via cross-contract calls to fetch boards and rosters before resolving a fight.

3. Contract State

3.1 PlayerState , Board contract

Field	Type
status	PlayerStatus
seed	Option<Vec<u8>>
shop_offer	Option<Vec<u8>>
board	Option<Vec<u8>>
games_played	u8
games_won	u8
season_id	Option<u32>

3.2 Battle, Battle contract

Field	Type
id	String
a_units / b_units	Vec<Unit>
a_health / b_health	i32
a_shield / b_shield	u32
a_fire / b_fire	u32
tick	u32
random_seed	Vec<u8>
status	BattleStatus
roster	Vec<UnitDef>

3.3 public methods

Method	Contract	Purpose
roll_seed(season_id)	Board	Payable (1 NEAR). Sets seed, generates shop offer via Fisher-Yates
lock_board(chosen_ids)	Board	Validates IDs were in shop offer; sets status to Ready
get_board(player)	Board	Cross-contract view; returns PlayerState for battle contract
load_roster(season_id)	Board	Returns Vec<UnitDef> for the given season
set_game_played(player)	Board	Increments games_played; triggers payout after 3 games
start_battle(opponent)	Battle	Fires parallel fetch to both boards; chains to roster fetch
on_boards_loaded()	Battle	Private callback; validates boards; stores PendingBattle; fetches roster
on_roster_loaded()	Battle	Private callback; builds boards; calls resolve_battle()
create_season(id, name, roster)	Board	Admin: creates a new season with its own unit pool

4. Asynchronous Call Flow

Step	Function	Description
1	Player → start_battle(opponent)	Player initiates battle; battle contract fires parallel calls
2	Battle → get_board(A) .and(get_board(B))	Fetch both player states in parallel from board registry
3	#[private] on_boards_loaded()	On call back env::promise_result match ;store PendingBattle; fire roster fetch
4	Battle → load_roster(season_id)	Fetch unit definitions for the season from board registry
5	#[private] on_roster_loaded()	On call back env::promise_result; build boards; call resolve_battle()
6	resolve_battle() → BATTLE_LOG	Tick loop runs; emits JSON log; returns winner status
7	Battle → set_game_played(A,B) + set_games_won(winner)	Update player stats on board registry; trigger payout if series complete

5. Access control model

Access control is enforced in 4 ways these are:

Gate	Functions	Enforcement
Admin (owner_id)	add_unit_to_season, create_season, set_active_season, enable/disable_unit	assert predecessor_account_id() == self.admin
Battle Contract	set_game_played, set_games_won,	assert predecessor_account_id() == self.battle_contract
Player Status Enum	lock_board (HasShop), get_board (Ready), pick_upgrade (AtBazaar)	PlayerStatus enum checked at function entry; panics on wrong state
#[payable]	roll_seed	assert attached_deposit == 1 NEAR; rejects underpaid calls

This prevents malicious usage of functions and mistakes by users

6. Storage deposit handling

For this Dapp I use the players entry deposit to the to the game, as players attach 1 NEAR when they generate the seed for their shop.

The current implementation of the payout is

Wins	Payout	Net
1 / 3	0.4 NEAR	-0.6 NEAR
2 / 3	1.0 NEAR	±0 NEAR
3 / 3	1.5 NEAR	+0.5 NEAR

Which means if all players finished all their matches there will be leftover NEAR on the contract for storage costs.

7. Future Extensions

Adding in the ability for players to make choices after a battle would be an interesting extension. As an example, the player gets 3 choices of upgrades for a unit that allows them to do increased damage. This would require adding in some additional “bazaar” that the player gets choices from and implement the code to apply these upgrades.

In conjunction with the other extension another idea is that the order of unit's matter, this could be done with units having an ability like “The unit to the right gets +1 damage”. This would add more complexity to the players board structure. To do this the battle simulation would need to be rewritten to allow checking of units left and right and new abilities that perform this action.